WORKSHOP-2 · ETL G01 · DATA ENGINEERING & AI · 2026-2

A reliable batch pipeline for Spotify × Grammy data

Two heterogeneous sources, two validation gates, one trusted star-schema Data Warehouse, and every failure anticipated, controlled and observable.

AIRFLOW 3.1.8 · AIRFLOW.SDK | GREAT EXPECTATIONS 1.X | POSTGRESQL 16 | METABASE | DOCKER COMPOSE

> “A Data Engineer does not build scripts.
> A Data Engineer designs systems.”

2

sources: Spotify **CSV** + Grammys from a **PostgreSQL** source DB

18

quality rules in **3 GX gates** (31 expectations)

113,549

fact rows loaded = prepared rows (full reconciliation)

4/4

reliability tests in Airflow: success · controlled failure · safe rerun · retry

§6.1 · START FROM THE DECISION, NOT THE DATA

Three questions that neither source can answer alone

ID	ANALYTICAL QUESTION → DECISION	FROM SPOTIFY	FROM GRAMMYS	KPI	LEVEL OF DETAIL
RQ-01	**Do Grammy winners sound different and are they more popular?** → which sound profile to prioritise in award-oriented repertoire	danceability, energy, valence, acousticness, popularity	nominee, artist, winner	KPI-01 audio profile + popularity uplift	distinct track × award status
RQ-02	**In which genres is a Grammy associated with the largest popularity uplift?** → where post-award promotion pays off	track_genre, popularity	nominee, artist, winner	KPI-02 uplift per genre (n ≥ 5)	track × genre
RQ-03	**How did explicit content and collaborations evolve among awarded tracks?** → editorial and collaboration strategy	explicit, artists (credit list)	year, nominee, artist	KPI-03 % explicit / % collab per decade	awarded track × decade

Why both sources

Spotify has sound, popularity and genre but **no recognition**. The Grammys have recognition and year but **no audio or consumption data**. Every KPI combines one attribute from each source.

Scope

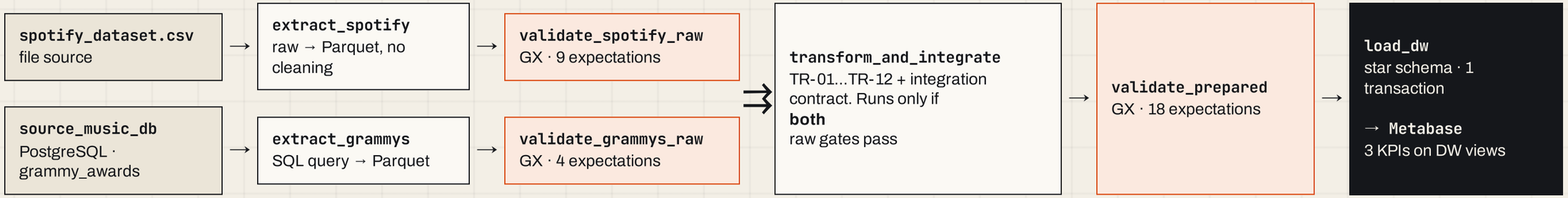
Batch, manually triggered; exact matching on a normalized key; association, not causation (popularity is one snapshot).

Traceability IDs

`RQ-xx` → `DQ-*` rules → `TR-xx` transformations → DW views → `KPI-xx`, used consistently in code, docs and evidence.

§5 · REQUIRED SOLUTION ARCHITECTURE, IMPLEMENTED AS-IS

Two observable branches, two gates, one gated load

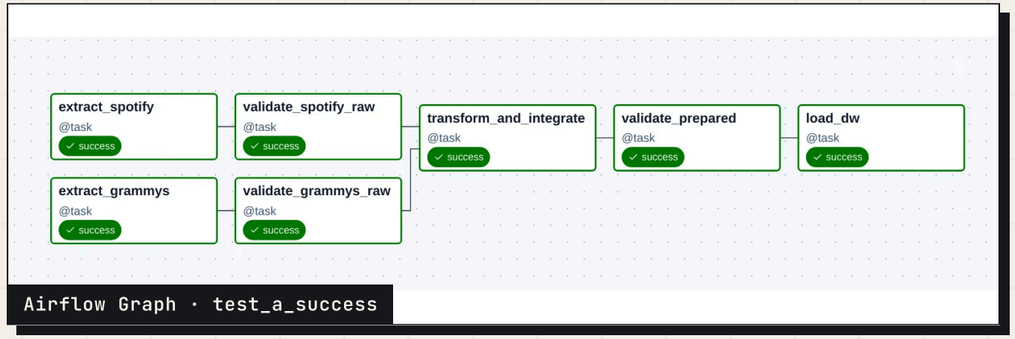


Orchestration

- `from airflow.sdk import dag, task` (TaskFlow)
- default `all_success`: load waits for the prepared gate
- `max_active_runs=1`, `schedule=None`

Interfaces

- XCom carries **paths + small dicts**, never DataFrames
- run-scoped staging `data/staging/<run_id>/`
- evidence `docs/evidence/runs/<run_id>/`



Airflow Graph · test_a_success

Source preparation ≠ ETL load

The `source-seed` service loads the provided Grammy CSV into PostgreSQL and reconciles it (**4,810 = 4,810 rows**). The DAG then extracts *only* from the database.

```python
# dags/reliable_music_pipeline.py (dependencies)
from airflow.sdk import dag, task, get_current_context

s_val = validate_spotify_raw_task(extract_spotify_task())
g_val = validate_grammys_raw_task(extract_grammys_task())
prep  = validate_prepared_task(
          transform_and_integrate_task(s_val, g_val))
load_dw_task(prep)
```

§6.3 · EVIDENCE BEFORE RULES · NOTEBOOKS/DATA_PROFILING.IPYNB (EXECUTED, OUTPUTS SAVED)

What the data actually looks like, and why it matters

450

duplicate `(track_id, track_genre)` rows (0.39%) → would double count genre KPIs → **DQ-RAW-SPT-06 · TR-03**

1

corrupted row: 0 ms duration, null title & artist → **DQ-RAW-SPT-02 · TR-01**

6

Grammy rows without `nominee` (0.12%) → cannot match → **DQ-RAW-GRM-02 (Warning) · TR-06**

38%

1,840 awards without `artist` (craft, classical, album categories) → structural coverage limit

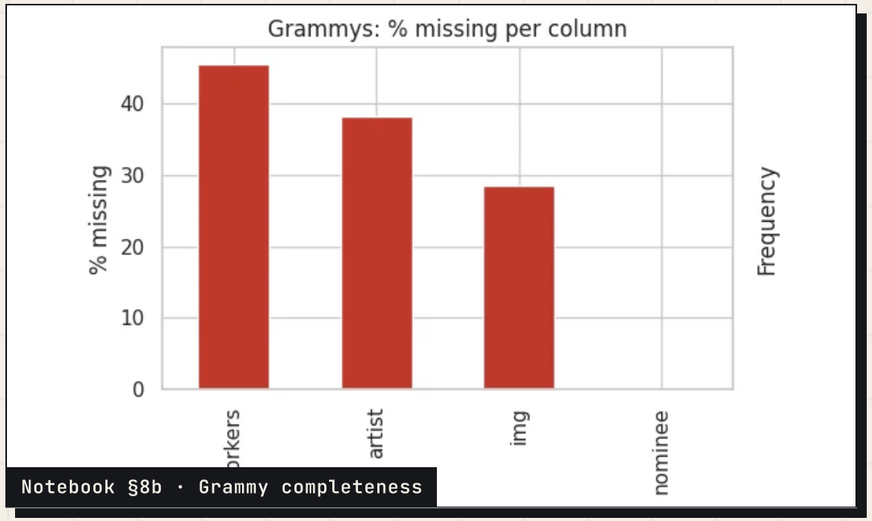
100%

4,810 / 4,810 Grammy rows are winners → no nominee tier, so we compare winners vs catalog

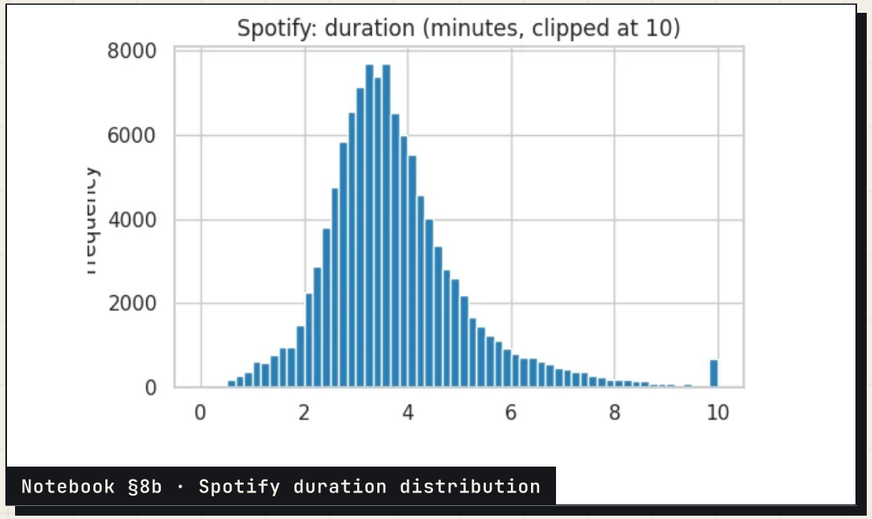
161→181

matched Grammy keys with raw text vs normalized key: `;` vs `feat.` / `&` → **TR-04/05**

Structure: Spotify 114,000 × 21, grain track × genre (89,741 distinct tracks). Grammys 4,810 × 10, years 1958–2019, key (year, category, nominee) 100% unique. Audio features within [0, 1], popularity within [0, 100].



Notebook §8b · Grammy completeness



Notebook §8b · Spotify duration distribution

§6.5 · 18 RULES · COVERAGE FOLLOWS RISK, NOT SYMMETRY

Rules with a dimension, a threshold and a consequence

RULE	LAYER	WHAT MUST BE TRUE	THRESHOLD	SEVERITY
SPT-05 · GRM-03 · PREP-03	all gates	Schema contract (required / exact columns)	100%	CRITICAL
RAW-SPT-01	Spotify raw	`track_id` not null	0 nulls	CRITICAL
RAW-SPT-02	Spotify raw	duration > 1,000 ms	≥ 99.9%	CRITICAL
RAW-SPT-03 · 04	Spotify raw	4 audio features ∈ [0, 1]; popularity ∈ [0, 100]	100%	CRITICAL
RAW-SPT-06	Spotify raw	(track, genre) unique	≥ 99%	WARNING
RAW-GRM-01	Grammy raw	year ∈ [1958, 2026]	100%	CRITICAL
RAW-GRM-02 · 04	Grammy raw	nominee populated · row-count band 4k–10k	≥ 99.5% · band	WARNING INFO
PREP-02 · 04	prepared	keys not null; ranges hold after transform	100%	CRITICAL
PREP-05	prepared	fact grain unique	0 duplicates	CRITICAL
PREP-06 · 07	prepared	matched ⇒ valid year & award_count ≥ 1; unmatched ⇒ year 0; both tiers exist	100%	CRITICAL
PREP-08 · 01	prepared	match rate plausible · flag is boolean	≤ 5% · set	WARNING INFO

13
Critical

3
Warning

2
Informational

Severity policy

CRITICAL → `CriticalDataQualityError`: task fails, downstream `upstream_failed`, DW untouched

WARNING → logged + recorded, `CONTINUE_WITH_WARNING`

INFO → recorded as a trend signal, never blocks

A threshold is a decision

Duration ≥ 99.9%: one corrupted row is tolerated (TR-01 removes it), but a systemic unit change (> 114 rows) stops the batch.

§6.6 · SRC/VALIDATION.PY · GX/EXPECTATIONS · GX/VALIDATION_DEFINITIONS · GX/CHECKPOINTS

Suite → Validation Definition → Checkpoint, for every rule

Batch Definition
staging Parquet of *this* run
(pandas source → asset)

+

Expectation Suite
each expectation tagged
`meta.rule_id` + `severity`

=

Validation Definition
data ↔ suite association

→

Checkpoint
controlled execution,
result_format SUMMARY

→

Severity policy
STOP ·
CONTINUE_WITH_WARNING ·
CONTINUE →
`<gate>_validation.json`

RUN	GATE	PASSED	DECISION
test_a_success	spotify_raw_suite	9 / 9	CONTINUE
test_a_success	grammys_raw_suite	4 / 4	CONTINUE
test_a_success	prepared_music_suite	18 / 18	CONTINUE
test_b_controlled…	grammys_raw_suite	3 / 4	STOP

Design choices

- One generic `run_gate()`: same chain for the 3 gates
- Ephemeral context per task: the two raw gates run in parallel with no shared-file race; config exported to `gx/`
- Results are per run and never overwritten, so success *and* failure evidence coexist

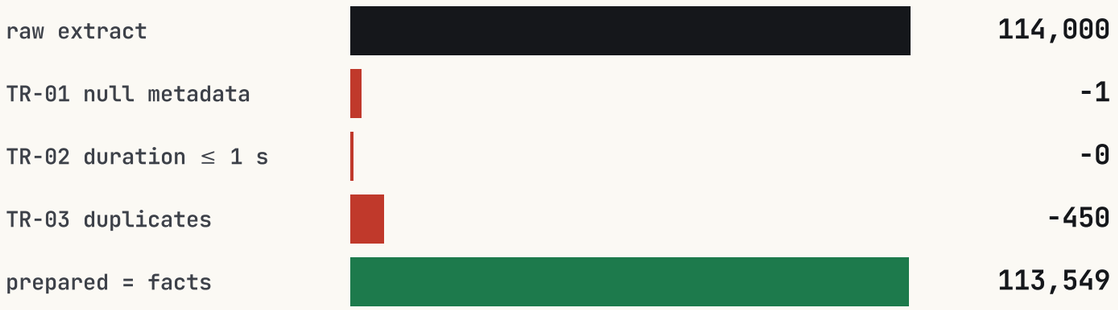
```
// docs/evidence/runs/test_b_controlled_failure/grammys_raw_validation.json
{ "suite": "grammys_raw_suite",
  "validation_definition": "grammys_raw_validation_definition",
  "checkpoint": "grammys_raw_checkpoint",
  "batch_context": { "run_id": "test_b_controlled_failure",
                     "source": "source_music_db.grammy_awards" },
  "gate_decision": "STOP",
  "summary": { "expectations_passed": 3, "critical_failures": ["DQ-RAW-GRM-01 (year)"] },
  "rules_evaluated": [ {
      "rule_id": "DQ-RAW-GRM-01", "severity": "Critical",
      "expectation_type": "expect_column_values_to_be_between",
      "success": false, "element_count": 4811,
      "unexpected_count": 1, "partial_unexpected_list": [1800] }, … ] }
```

Mapping: every one of the 31 expectations → Rule ID in `docs/quality_rules.md §3.1`.
Examples: `expect_compound_columns_to_be_unique` → PREP-05;
`row_condition="is_grammy_winner==True"` → PREP-06.

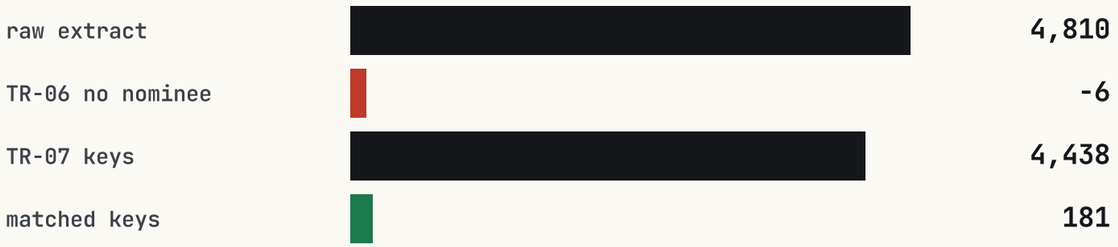
§6.8 · DOCS/TRANSFORMATION_DECISIONS.MD · DOCS/INTEGRATION_CONTRACT.MD

Integration is a contract, not “perform a merge”

Spotify reconciliation (integration_report.json)



Grammy side



Key	(normalized title, normalized primary artist). TR-04: lower, trim, punctuation → space. TR-05: first artist before `;` / `feat.` / `&` / `with`
Cardinality	**many-to-one**, enforced with `merge(validate="many_to_one")` : Grammy rows are aggregated per key first (286 keys with > 1 award)
Join	LEFT from the catalog: the non-awarded tracks are the comparison group for RQ-01/02
Unmatched	Spotify: `is_grammy_winner=False` , default members SK 0 (TR-09). Grammys: **4,257 keys (95.9%)** counted, not loaded
Duplicates	0 per Spotify row by design; 49 keys hit > 1 track_id (re-releases, max 9), so they are kept and measured

475
matched fact rows

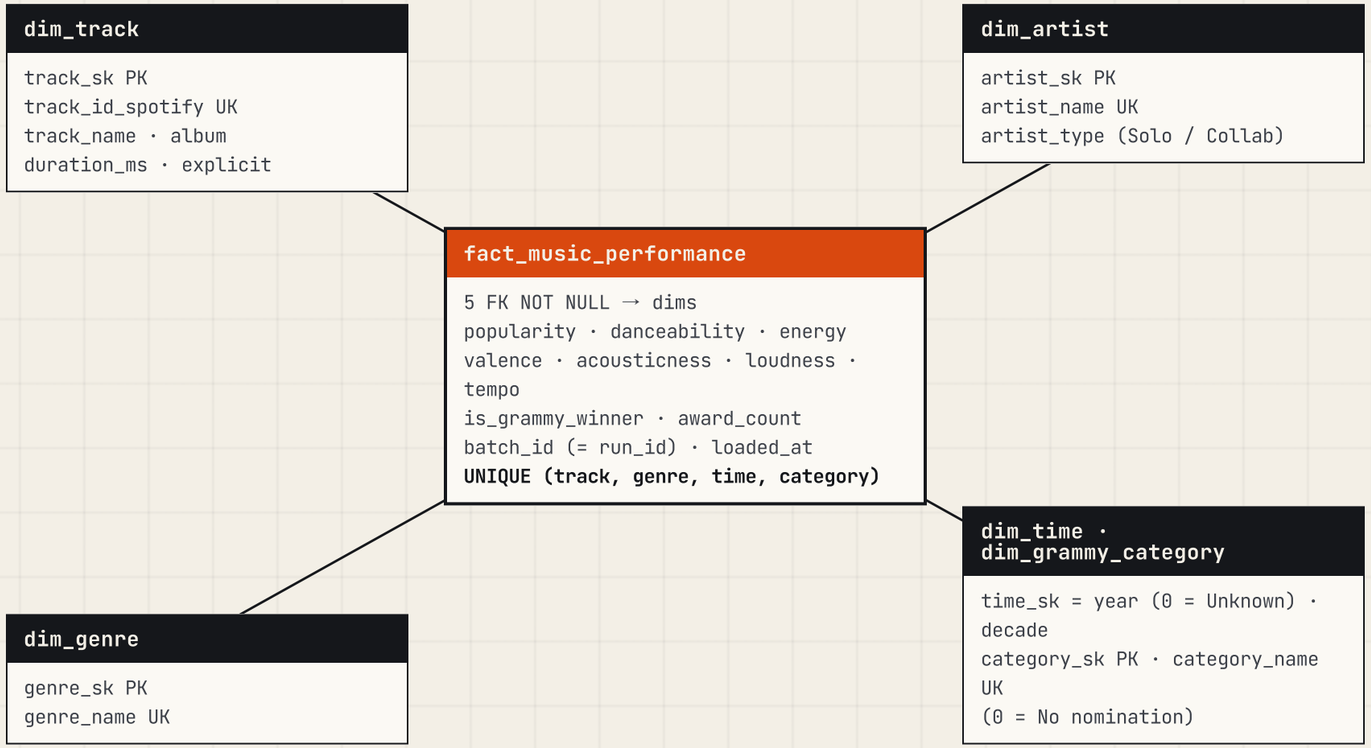
282
distinct awarded tracks

0.42%
match rate (Warning if > 5%)

> **Rule of the phase:** each TR-xx is justified by profiling or a requirement, never by “so the validation passes”. Values that cannot be repaired are dropped and counted, never imputed.

§6.4 · §6.10 · SQL/DW_SCHEMA.SQL (IDEMPOTENT) · DOCS/DIMENSIONAL_MODEL.MD

A requirements-driven star schema in PostgreSQL



Grain

One row per **track × genre × award year × award category**. Unmatched tracks use default members (SK 0), so there are no NULL foreign keys.

REQUIREMENT	DW ELEMENT
RQ-01 audio + popularity	`vw_track_profile` → `vw_kpi1_audio_profile`
RQ-02 uplift by genre	`vw_kpi2_genre_uplift` (with n)
RQ-03 decade trends	`vw_kpi3_decade_trends`

- Surrogate keys + `UNIQUE` business keys, used by the rerun upserts
- `CHECK` constraints mirror the GX ranges
- Track-level KPIs collapse genres first, so a song counts once

113,549 facts

89,740 tracks

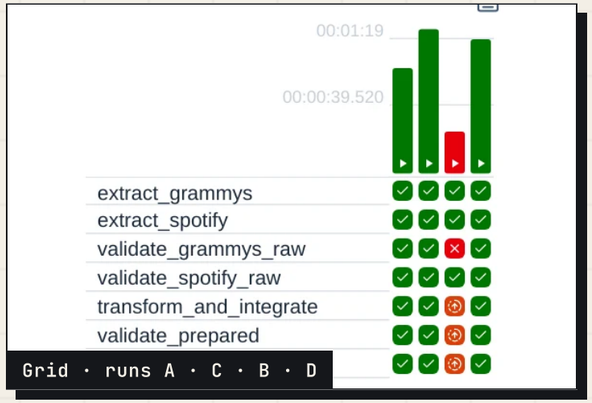
17,645 artists

§6.11 · THE HIGHEST-WEIGHTED CRITERION

Retries only where another attempt can succeed

TASK CLASS	FAILURE TYPE	RETRIES	WHY
`extract_spotify` `extract_grammys`	Transient: I/O, DB refusing connections	`2 · 15 s · exp. backoff (≤ 2 min)`	Same input + same code can succeed later. **Proved in Test D.**
`validate_*` (3 gates)	Deterministic data / contract	`0`	The same batch fails the same way. Fail fast with evidence. **Test B.**
`transform_and_integrate`	Deterministic code / cardinality	`0`	A traceback is reproducible; retrying hides nothing.
`load_dw`	Transient: lock timeout, connection drop	`1 · 30 s`	One atomic transaction, so a retry is safe; a single attempt limits repeated heavy loads.

`DEFAULT_ARGS = {"retries": 0}`: there is no blanket `retries=3`. Each task opts in according to its failure class. Full condition table in `docs/architecture.md §3`.



Reading the grid

Columns are runs A, C, B and D. In run B, `validate_grammys_raw` is red and the three downstream cells are orange (`upstream_failed`). In run D everything is green, even though `extract_grammys` needed two tries.

Observability: every task log starts with

```
BATCH CONTEXT | dag=reliable_music_pipeline
run_id=test_d_transient_retry task=extract_grammys try=2
```

What ran?	run_id, task, try in every log line
When?	task start / end (UI + task_states.txt)
Succeeded?	states + material counts (rows in / removed / matched / inserted / updated)
Where & why?	red task + `upstream_failed`; error names the rule and the JSON path
What next?	`up_for_retry` → try 2, or stop on Critical

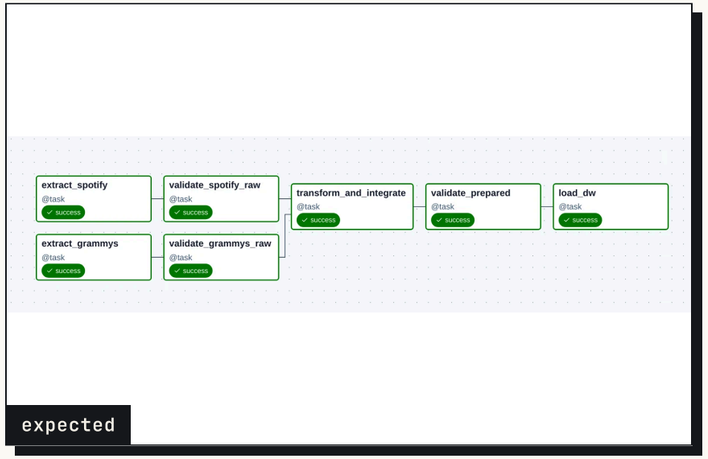
PRESENTATION RUBRIC · DEMO · FALLBACK EVIDENCE ON THE NEXT SLIDES

Live demo: success, controlled failure, safe rerun

1 · Successful run

```
docker compose exec airflow \
  airflow dags trigger \
  reliable_music_pipeline
```

Show the Grid: 7 green tasks. Open `load_dw` log → reconciliation summary.

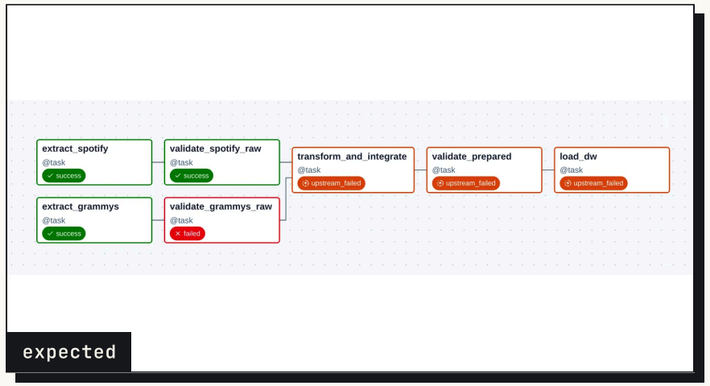


expected

2 · Controlled Critical failure

```
INSERT INTO grammy_awards
 (year, title, category,
  nominee, artist, winner)
VALUES (1800, …, TRUE);
```

Trigger → `validate_grammys_raw` red, three tasks `upstream_failed`. Open the log → DQ-RAW-GRM-01 + JSON path.

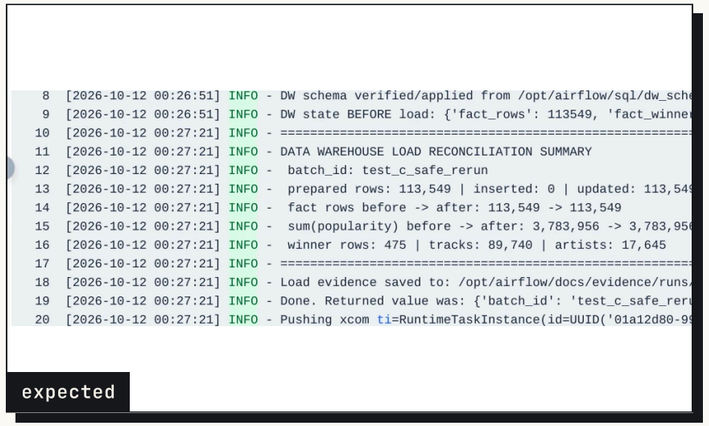


expected

3 · Recover & rerun

```
DELETE FROM grammy_awards
 WHERE year = 1800;
-- trigger again, then:
SELECT COUNT(*) …
```

Same 113,549 rows and same `SUM(popularity)`; only `batch_id` changes.

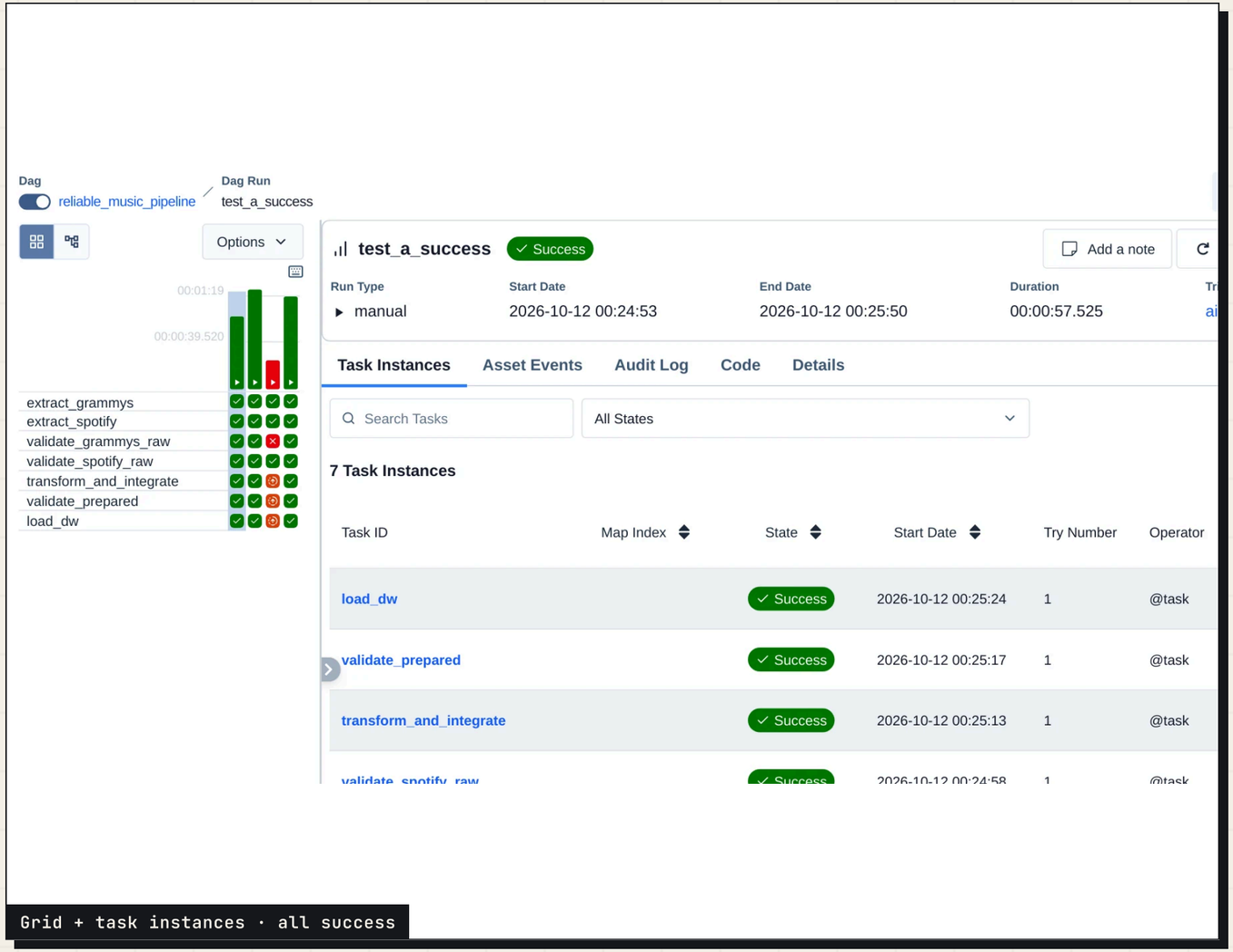


expected

Everything is reproducible: `bash scripts/run_reliability_tests.sh` runs A → C → B → D in about 6 minutes and regenerates the evidence. If the live environment fails, the screenshots on the next four slides come from exactly that script.

TEST A · RUN TEST_A_SUCCESS · 2026-10-12 00:24:53 → 00:25:50 UTC

Every stage succeeds, and every row is accounted for



Grid + task instances · all success

7/7
tasks success · ≈ 57 s

31/31
expectations passed (9 · 4 · 18)

113,549
inserted = prepared rows

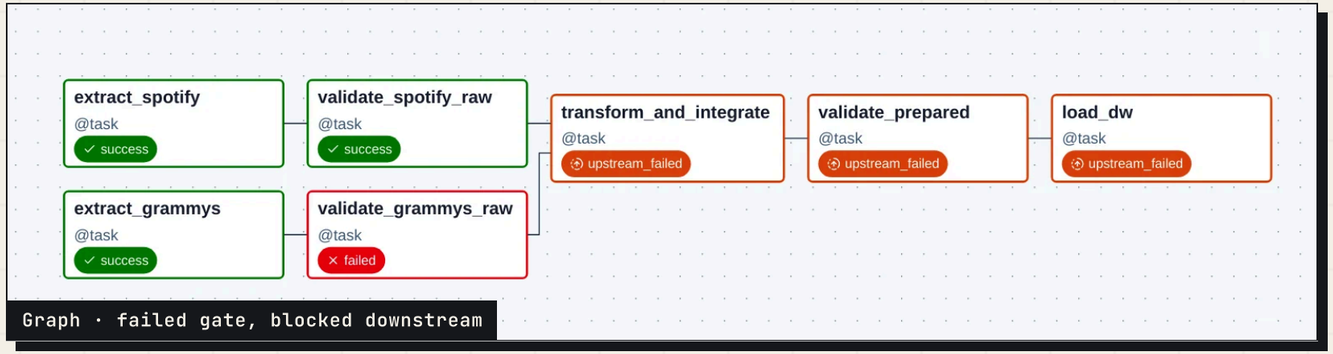
475
winner rows · 282 tracks

```
 8  [2026-10-12 00:25:25] INFO - DW schema verified/applied from /opt/airflow/sql/dw_sch
 9  [2026-10-12 00:25:25] INFO - DW state BEFORE load: {'fact_rows': 0, 'fact_winner_rows
10  [2026-10-12 00:25:49] INFO - ============================================================
11  [2026-10-12 00:25:49] INFO - DATA WAREHOUSE LOAD RECONCILIATION SUMMARY
12  [2026-10-12 00:25:49] INFO -  batch_id: test_a_success
13  [2026-10-12 00:25:49] INFO -  prepared rows: 113,549 | inserted: 113,549 | updated: 0
14  [2026-10-12 00:25:49] INFO -  fact rows before -> after: 0 -> 113,549
15  [2026-10-12 00:25:49] INFO -  sum(popularity) before -> after: 0 -> 3,783,956
16  [2026-10-12 00:25:49] INFO -  winner rows: 475 | tracks: 89,740 | artists: 17,645
17  [2026-10-12 00:25:49] INFO - ============================================================
18  [2026-10-12 00:25:49] INFO - Load evidence saved to: /opt/airflow/docs/evidence/runs/
    [2026-10-12 00:25:49] INFO - Done. Returned value was: {'batch_id': 'test_a_success'}
    [2026-10-12 00:25:49] INFO - Marking task as SUCCESS. dag_id=reliable_music_pipeline, task_id=load_dw, run_id=manual__2026-10-12T00:24:53+00:00, map_index=-1, execution_date=20261012T002453, start_date=20261012T002525, end_date=20261012T002549
    [2026-10-12 00:25:49] INFO - Task exited with return code 0
    [2026-10-12 00:25:49] INFO - 0 downstream tasks scheduled from follow-on schedule check
```

load_dw log · reconciliation summary

TEST B · RUN TEST_B_CONTROLLED_FAILURE · RULE DQ-RAW-GRM-01 [CRITICAL]

An invalid source year stops the pipeline before it does harm



Graph · failed gate, blocked downstream

```
223  [2026-10-12 00:28:21] INFO - [grammys_raw] Validation evidence saved to: /opt/airflow/docs/evi
224  [2026-10-12 00:28:21] INFO - [grammys_raw] 3/4 expectations passed on 4,811 records -> gate d
225  [2026-10-12 00:28:21] ERROR - Critical quality gate failure in grammys_raw: DQ-RAW-GRM-01 (yea
226  [2026-10-12 00:28:21] ERROR - Task failed with exception
                                   ▼ CriticalDataQualityError: Critical quality gate failure in
                                   File "/home/airflow/.local/lib/python3.11/site-packages/airf
                                   File "/home/airflow/.local/lib/python3.11/site-packages/airf
```

validate_grammys_raw log · actionable cause

```
// runs/test_b_controlled_failure/grammys_raw_validation.json
"gate_decision": "STOP",
"rule_id": "DQ-RAW-GRM-01", "severity": "Critical",
"element_count": 4811, "unexpected_count": 1,
"partial_unexpected_list": [1800]
```

STAGE	EXPECTED	OBSERVED
`extract_grammys`	success	SUCCESS 4,811 rows, invalid row not hidden
`validate_grammys_raw`	failure	FAILED no retry (retries=0)
`transform_and_integrate`	does not proceed	UPSTREAM_FAILED
`validate_prepared` · `load_dw`	do not proceed	UPSTREAM_FAILED
Spotify branch	unaffected	SUCCESS

Why this rule, why this response

1958 is the first ceremony. A year like 1800 would create a false time member and distort the RQ-03 decade trend, so loading is unsafe. It is deterministic, so retrying cannot fix it.

DW protected

After the failed run: still **113,549** rows, `batch_id = test_c_safe_rerun` (previous good batch). Recovery: fix the source, trigger again.

TEST C · RUN TEST_C_SAFE_RERUN · SAME INPUT BATCH AS TEST A

Same batch twice: nothing changes but the audit stamp

METRIC	BEFORE	AFTER	Δ
fact rows	113,549	113,549	0
grain duplicates	0	0	0
SUM(popularity)	3,783,956	3,783,956	0
AVG(danceability)	0.567031	0.567031	0
SUM(award_count)	538	538	0
winner rows	475	475	0

0
inserted

113,549
updated

0
purged

load_dw log · rerun

```
[2026-10-12 00:26:51] INFO - DW schema verified/applied from /opt/airflow/sql/dw_sch
[2026-10-12 00:26:51] INFO - DW state BEFORE load: {'fact_rows': 113549, 'fact_winne
[2026-10-12 00:27:21] INFO - ============================================================
[2026-10-12 00:27:21] INFO - DATA WAREHOUSE LOAD RECONCILIATION SUMMARY
[2026-10-12 00:27:21] INFO -  batch_id: test_c_safe_rerun
[2026-10-12 00:27:21] INFO -  prepared rows: 113,549 | inserted: 0 | updated: 113,54
[2026-10-12 00:27:21] INFO -  fact rows before -> after: 113,549 -> 113,549
[2026-10-12 00:27:21] INFO -  sum(popularity) before -> after: 3,783,956 -> 3,783,95
[2026-10-12 00:27:21] INFO -  winner rows: 475 | tracks: 89,740 | artists: 17,645
[2026-10-12 00:27:21] INFO - ============================================================
[2026-10-12 00:27:21] INFO - Load evidence saved to: /opt/airflow/docs/evidence/runs/
[2026-10-12 00:27:21] INFO - Done. Returned value was: {'batch_id': 'test_c_safe_rer
[2026-10-12 00:27:21] INFO - Pushing xcom ti=RuntimeTaskInstance(id=UUID('01a12d80-9
```

```
-- strategy: snapshot load in ONE transaction
CONSTRAINT uq_fact_grain UNIQUE (track_sk, genre_sk, time_sk, category_sk)

INSERT INTO fact_music_performance (...) SELECT ...
ON CONFLICT (track_sk, genre_sk, time_sk, category_sk)
DO UPDATE SET popularity = EXCLUDED.popularity, ...,
              batch_id = EXCLUDED.batch_id
RETURNING (xmax = 0) AS inserted;      -- inserted vs updated

DELETE FROM fact_music_performance
 WHERE batch_id <> :run_id;            -- stale rows of older snapshots
-- rows written ≠ prepared rows  →  RuntimeError → ROLLBACK
```

Why upsert + purge (not upsert alone)

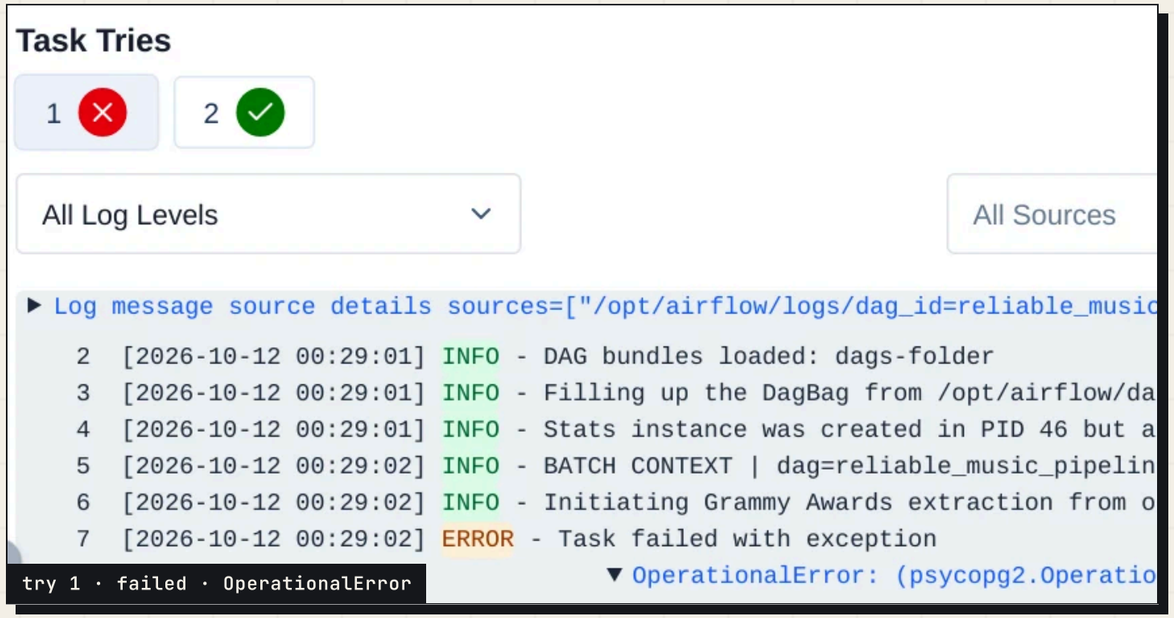
If a track changes from unmatched to matched, its grain key changes. An upsert alone would keep the old row next to the new one. The purge removes rows that are not in the current snapshot.

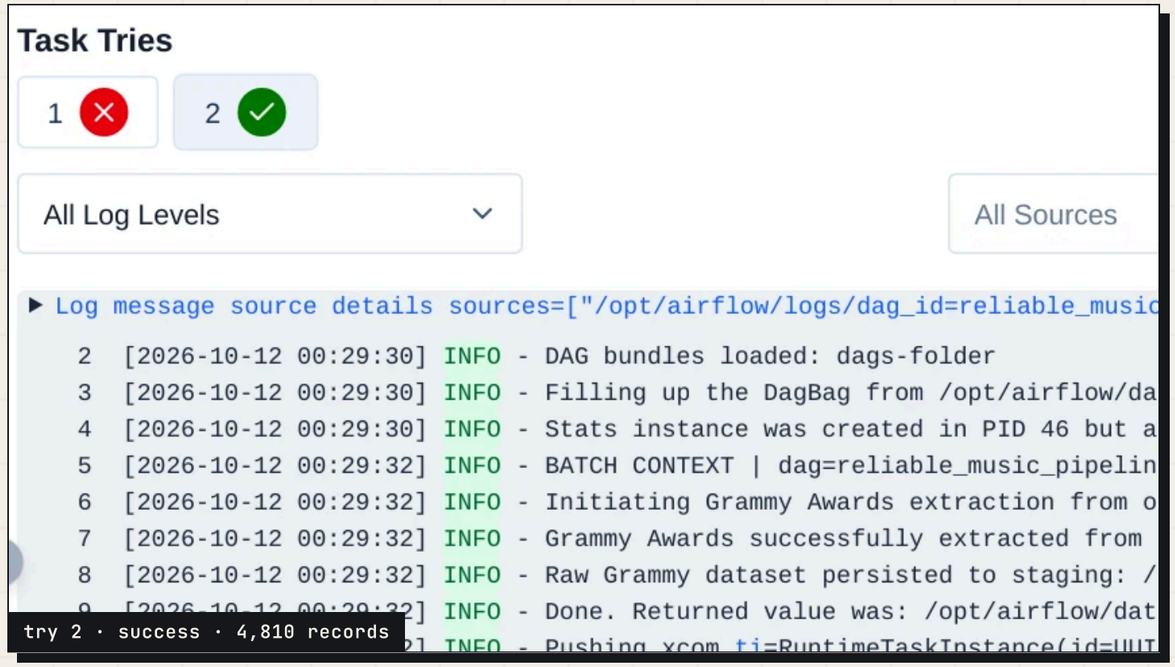
Partial or failed runs

Dimensions and facts share one `engine.begin()` transaction: any error rolls everything back and the previous snapshot stays. A failure before the load (Test B) never touches the DW.

TEST D · RUN TEST_D_TRANSIENT_RETRY · SOURCE DB REFUSES NEW CONNECTIONS

A transient outage is retried and recovers on its own





00:28:54

`ALTER DATABASE source_music_db ALLOW_CONNECTIONS false`

00:29:01 · try 1

`database "source_music_db" is not currently accepting connections` → **up_for_retry**

00:29:12

connections restored (outage over)

00:29:30 · try 2

4,810 rows extracted → the remaining 6 tasks succeed; load is idempotent (0 inserted)

§8.1 · METABASE ON DW VIEWS · NEVER ON CSV

What the trusted data says



Metabase · Workshop-2 KPIs (DW)

+12.04
popularity pts

KPI-01 · RQ-01

Winners 45.21 vs catalog 33.17 (282 vs 89,458 tracks). Higher valence (0.53 vs 0.47), lower acousticness (0.26 vs 0.33).

+45.6
soul (n = 20)

KPI-02 · RQ-02

Then jazz +42.4 (n=9) and country +23.6 (n=26); negative in pop −21.3 and electro −16.2. Genres need ≥ 5 awarded tracks to be compared, and n is always shown.

20.2%
explicit, 2010s

KPI-03 · RQ-03

0% until the 1980s → 7.3% → 14.1% → 20.2%. Collaborations jump to 35.7% in the 2010s (n = 84 tracks).

Query path: Metabase SQL → `vw_kpi1/2/3` → star schema. Built by `scripts/setup_metabase.py`. Metabase needs instructor approval; the same views work in Tableau (`docs/bi_tableau_guide.md`).

§8.2 · DOCS/TRACEABILITY_MATRIX.MD

Every KPI can be traced back to a requirement, a risk and a rule

	REQUIRED DATA	PROFILING RISK	DQ RULE	GX EXPECTATION	TRANSFORMATION	DW ELEMENT	KPI
RQ-01	Spotify title / artists + Grammy nominee / artist	`;` vs `feat.`: raw text matches 161 keys	PREP-07 Critical · PREP-08 Warning	distinct_values_to_contain_set · values_in_set(mostly .95)	TR-04 / 05 / 07	`is_grammy_winner` → vw_kpi1	+12.04 pts uplift
RQ-02	Spotify (track_id, genre), popularity	450 duplicates would double count	SPT-06 Warning · PREP-05 Critical	compound_columns_to_be_unique	TR-03 · TR-11	`uq_fact_grain` → vw_kpi2	uplift by genre with n
RQ-03	Grammy year · Spotify explicit, artists	out-of-range year breaks dim_time	GRM-01 Critical	values_to_be_between(1958, 2026)	TR-10 decade · TR-08 artist_type	`dim_time` → vw_kpi3	20.2% explicit (2010s)
ALL	matched / unmatched award fields	joins could leave rows half-filled	PREP-06 Critical	values_to_be_between + row_condition	TR-09 default members	SK 0 in dim_time / category	winner vs catalog split

Reasoning chain (§4.1)

requirement → data → profiling → risk → dimension → rule → threshold & severity → expectation → result → pipeline action.

Example, end to end

RQ-03 needs years → profiling says 1958–2019 → rule GRM-01 [1958, 2026] Critical → violated in Test B → **action: stop** before dim_time is corrupted.

Consistent IDs

The same IDs appear in code comments, the GX `meta.rule_id`, the JSON results, the docs and the evidence register.

§9 · GITHUB.COM/MACOWLI-0611/WORKSHOP2ETL

Clone, configure one file, run one command

```
cp .env.example .env        # set passwords
docker compose up -d --build
# postgres · airflow-init · source-seed
# (Grammy source + reconciliation) · airflow · metabase

bash scripts/run_reliability_tests.sh   # A · C · B · D
docker compose exec airflow python \
  /opt/airflow/scripts/setup_metabase.py
```

Verified from zero

The volume was deleted and the stack rebuilt: the schema bootstraps itself (idempotent DDL), and the source seeds and reconciles automatically.

Security hygiene

- Secrets only in `.env` (git-ignored); `.env.example` has placeholders
- Password found in old git history → rotated
- All ports bound to 127.0.0.1

```
dags/reliable_music_pipeline.py      TaskFlow DAG (wiring only)
src/  extract · validation · transform · load
sql/  source_setup · dw_schema (+ KPI views) · dw_reset
gx/   expectations · validation_definitions · checkpoints
notebooks/data_profiling.ipynb       executed, outputs saved
scripts/  seed · reliability tests · metabase · charts
          log export · screenshots
docs/ requirements · architecture · quality_rules
      transformation_decisions · integration_contract
      dimensional_model · traceability · dashboard
docs/evidence/
      evidence_register.md           EV-01 … EV-17
      runs/<run_id>/*.json            GX · integration · load
      test_a… test_d/                 reports · logs · screenshots
```

EVIDENCE	ARTIFACT	PROVES
EV-02	extract_grammys log	Grammys come from PostgreSQL, not the CSV
EV-09	grammys_raw_validation.json (B)	failure linked to DQ-RAW-GRM-01
EV-12	dw_before / dw_after (C)	no duplicates, identical measures
EV-13	try_history.txt (D)	selective retry recovers

README covers all 17 mandatory sections; `docs/evidence/evidence_register.md` lists EV-01…EV-17.

WHAT THE SYSTEM CAN AND CANNOT ESTABLISH

Honest limits, measurable guarantees

Limitations

- **Winners only**: no nominee tier in the source
- **Exact match**: remasters, live versions and missing artists stay unmatched (4.1% of Grammy keys matched), measured every run
- **One popularity snapshot**: association, not causal post-award effect
- **Small n** in some genres: shown next to every value
- Award years end in 2019

Next steps

- Second-pass fuzzy match into a **review queue**, never straight into the DW
- A nominee source to enable a winner / nominee / catalog tier
- Scheduled runs with alerting on Critical failures
- Unit tests for TR rules in CI

RUBRIC CRITERION	PTS	WHERE WE PROVE IT
Requirements & architecture	0.40	slides 2–3 · traceability 16
Profiling & quality risks	0.60	slides 4–5 · executed notebook
Transformation & integration	0.60	slide 7 · integration_report.json
Great Expectations	0.80	slide 6 · per-run JSON · gx/
Airflow & reliability	0.90	slides 9–14 · Tests A–D
Dimensional DW & analytics	0.40	slides 8, 15 · KPI views
Repository & documentation	0.30	slide 17 · README · evidence register

> Reliable does not mean nothing fails. It means failures are anticipated, controlled and observable.

BACKUP · LIKELY QUESTIONS

Short answers to hard questions

Why are validation tasks not retried?

Data-quality failures are deterministic: the same batch reproduces the same violation. Retrying only delays the evidence (Test B). Transient classes do retry (Test D).

Validation vs transformation?

Validation measures and records; transformation applies rules that stand on their own (TR-xx). We never change data to make a check pass. Values that cannot be repaired are dropped and counted, not imputed.

Why a LEFT join and many-to-one?

Non-awarded tracks are the comparison group. Aggregating Grammy rows per key first makes duplicate matches impossible, and `validate="many_to_one"` enforces it.

Why is coverage so low? Is the integration wrong?

The prepared gate proves the integration is coherent (PREP-06/07). Low coverage is structural (38% without artist, non-song categories), and it is measured every run.

Why not pass DataFrames through XCom?

XCom lives in the Airflow metadata DB and is meant for coordination. Tasks pass Parquet paths and small dicts; data is persisted per run for investigation.

What if load_dw fails halfway?

One transaction: everything rolls back, and the previous snapshot stays. The retry or the next run converges because the load is idempotent.

How did you pick thresholds?

From profiling and the requirement they protect: e.g. duration ≥ 99.9% tolerates one corrupted row but stops a systemic error; match rate ≤ 5% flags a key that became too loose.

Why PostgreSQL and not SQLite?

The brief forbids SQLite. PostgreSQL gives transactions, `ON CONFLICT` upserts, `CHECK` constraints and concurrent access for Airflow, Metabase and the pipeline.